Fakultas Informatika
School of Computing
Telkom University

CAK2HAB3 - Dasar Kecerdasan Artifisial

Heuristic Search: Greedy BFS and A* Search

Lecturer Team

Outline

- Best-First Search
- A* Search

Best-First Search

Best-First Search

- use an evaluation function $f(n)$ for each node
  - $f(n)$ provides an estimate for the total cost.
  - Expand the node n with smallest $f(n)$.
- typically implemented using a priority queue.

Best-First Search Algorithm

- Initialize OPEN queue with initial state (node), and CLOSED set is empty
- Repeat until current state is goal or no more state in OPEN queue
  - Get the best node x from OPEN
  - If x is goal, return
  - Otherwise add x to CLOSED
  - Generate all successor for x, for all its successors y:
    - Evaluate y from parent x
    - If the successor has never been generated,
      - and add to OPEN, then save x as its parent
    - If the successor has already in OPEN but with different parent,
      - If the evaluation from x is better, change its parent and reevaluate all its successors

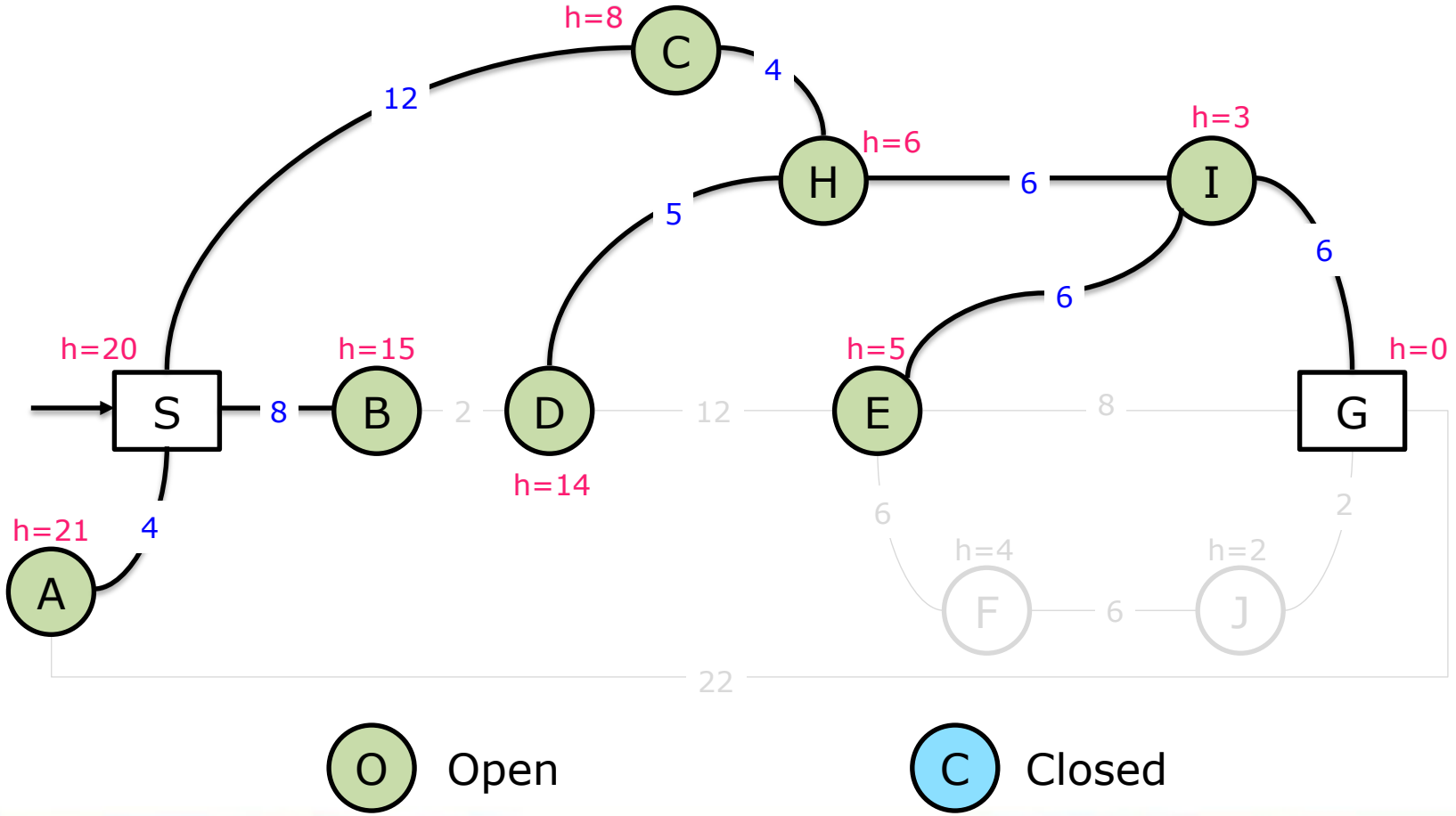
Greedy Best-First Search

- The simplest Best-First Search
- Only take into account the estimated cost
- Actual costs are not taken into account

$$f(n) = h(n)$$

- **Complete**, yet **not Optimal**

Greedy Best-First Search



Greedy Best-First Search Performance

- Complete
- Not Optimal
- Time Complexity = $O(b^m)$
- Space Complexity = $O(b^m)$

A* Search

A* Search

- Combining Uniform Cost Search and Greedy Best-First Search
  - avoid expanding paths that are already expensive
- The function calculated from the actual cost and the estimated cost

$$f(n) = g(n) + h(n)$$

- **Complete** and **Optimal**

A* Search Algorithm

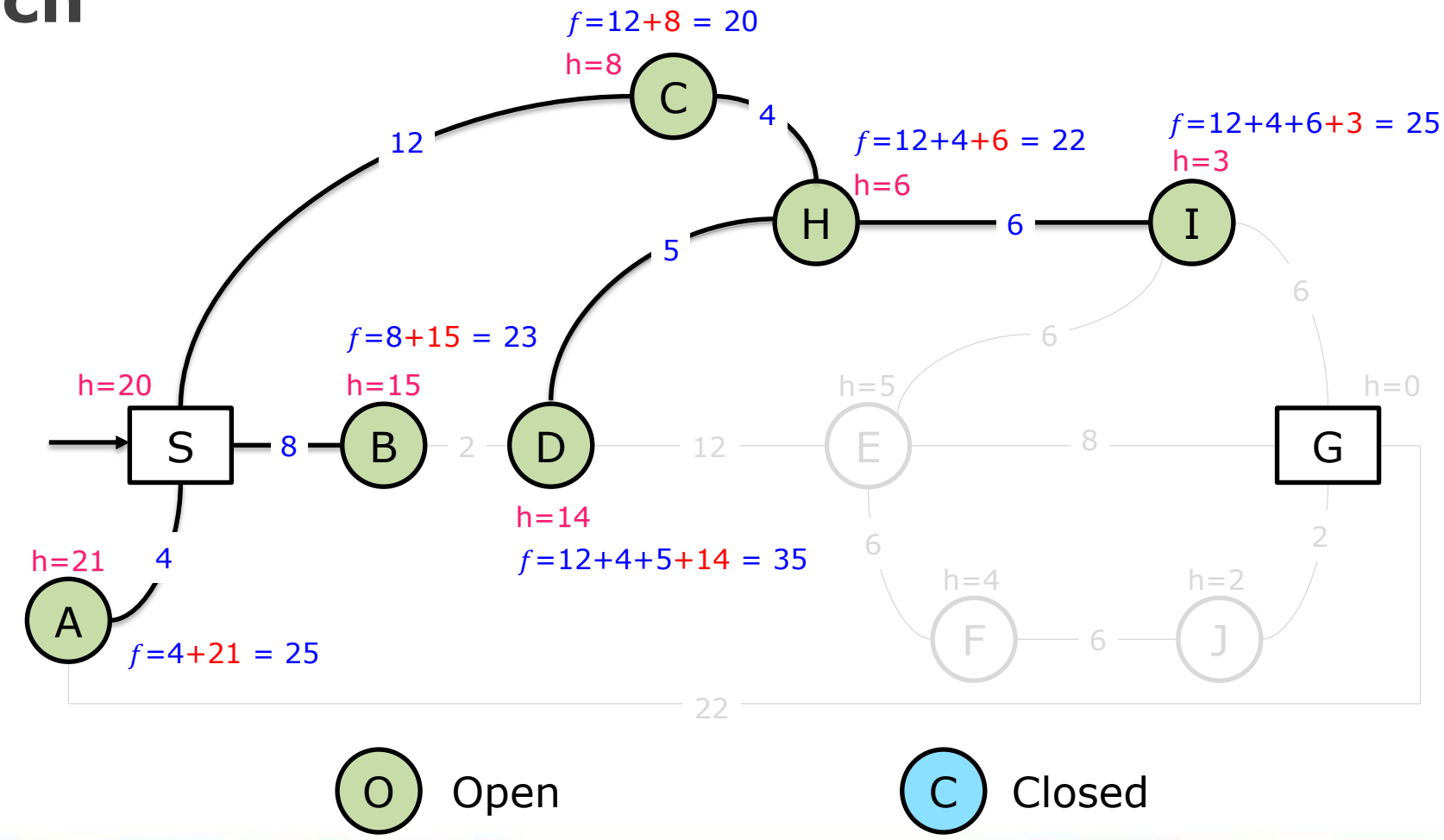
```
function reconstruct_path(cameFrom, current)
    total_path := [current]
    while current in cameFrom.Keys:
        current := cameFrom[current]
        total_path.append(current)
    return total_path
```

```
function A*(start, goal)
  closedSet := {}
  openSet := {start}
  cameFrom := the empty map
  gScore := map with default value of Infinity
  gScore[start] := 0
  fScore := map with default value of Infinity
  fScore[start] := heuristic_cost_estimate(start, goal)
```

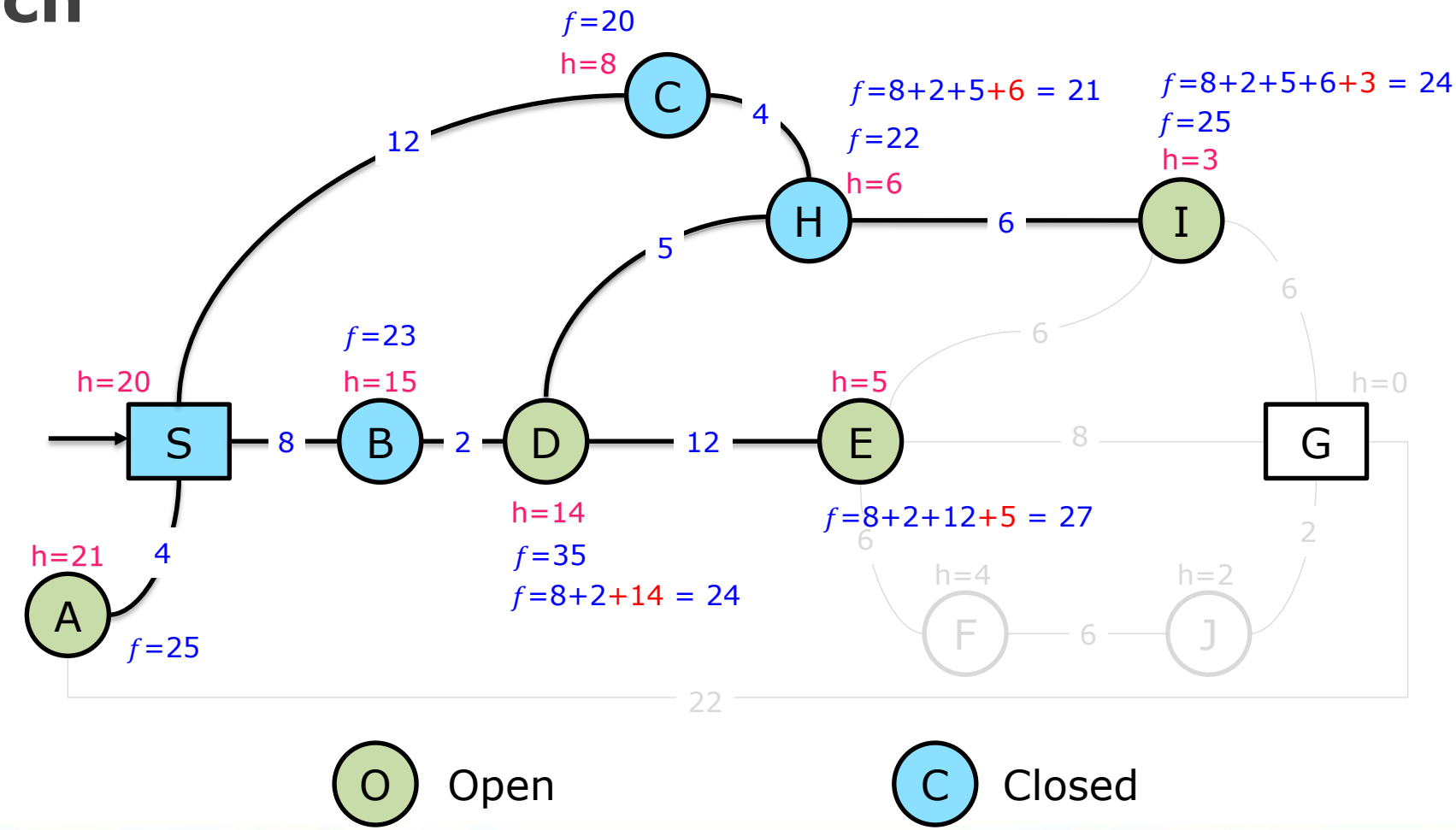
A* Search Algorithm

```
while openSet is not empty
  current := the node in openSet having the lowest fScore[] value
  if current = goal
    return reconstruct_path(cameFrom, current)
  openSet.Remove(current)
  closedSet.Add(current)
  for each neighbor of current
    if neighbor in closedSet
      continue     // Ignore the neighbor which is already evaluated.
    if neighbor not in openSet// Discover a new node
      openSet.Add(neighbor)
    tentative_gScore := gScore[current] + dist_between(current, neighbor)
    if tentative_gScore >= gScore[neighbor]
      continue     // not a better path.
    cameFrom[neighbor] := current
    gScore[neighbor] := tentative_gScore
    fScore[neighbor] := gScore[neighbor] + heuristic_cost_estimate(neighbor, goal)
return failure
```

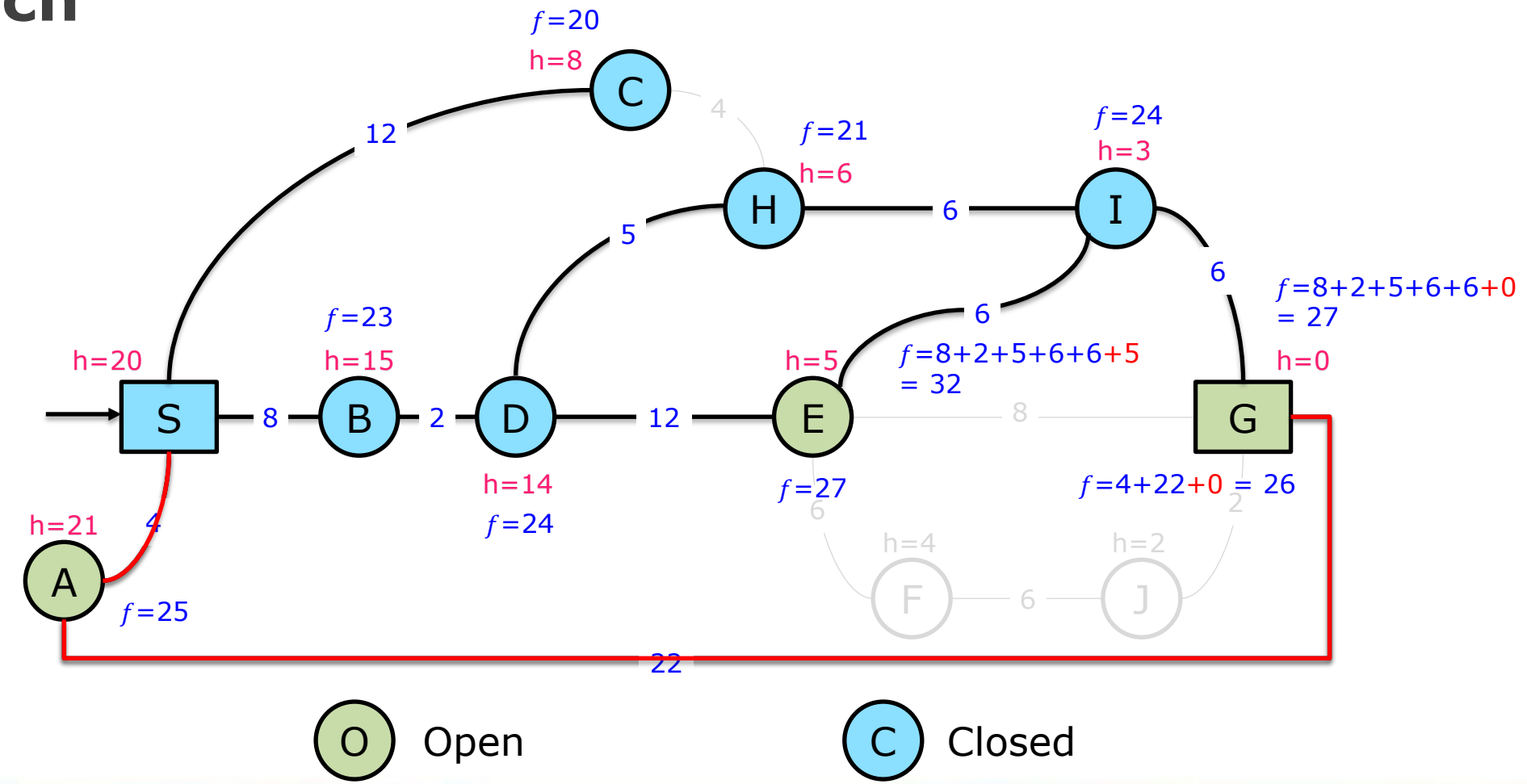
A* Search


f=12+8 = 20
h=8
C
4
12
f=12+4+6 = 22
h=6
H
6
I
f=12+4+6+3 = 25
h=3
5
f=8+15 = 23
h=15
h=20
S
8
B
2
D
12
E
8
G
h=5
h=0
6
6
h=14
f=12+4+5+14 = 35
h=21
4
A
f=4+21 = 25
6
2
h=4
h=2
F
6
J
22
O
Open
C
Closed

A* Search


f=20
h=8
C
4
f=8+2+5+6 = 21
f=22
h=6
f=8+2+5+6+3 = 24
f=25
h=3
12
H
6
I
5
6
6
f=23
h=15
h=20
h=5
h=0
S
8
B
2
D
12
E
8
G
h=14
f=8+2+12+5 = 27
f=35
f=8+2+14 = 24
6
2
h=21
4
h=4
h=2
A
F
6
J
f=25
22
O
Open
C
Closed

A* Search


f=20
h=8
C
12
4
f=21
h=6
H
6
f=24
h=3
I
5
6
6
f=8+2+5+6+6+0
= 27
h=0
G
f=23
h=15
h=20
S
8
B
2
D
12
E
h=5
f=8+2+5+6+6+5
= 32
8
h=14
f=24
f=27
f=4+22+0 = 26
6
2
h=4
h=2
F
6
J
h=21
4
A
f=25
22
O
Open
C
Closed

A* Search Performance

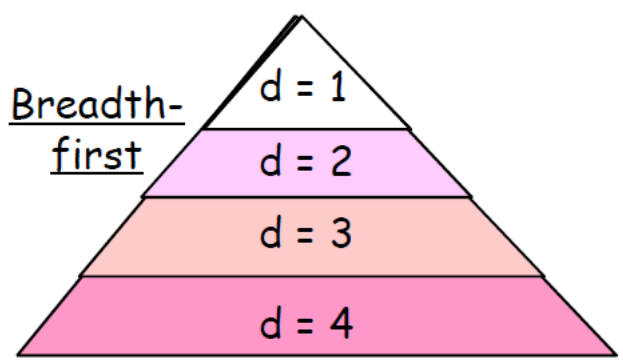
- Complete
- Optimal
  - No algorithm with the same heuristic is guaranteed to expand fewer nodes
- Time Complexity = $O(b^d)$
- Space Complexity =$O(b^d)$

More of A*

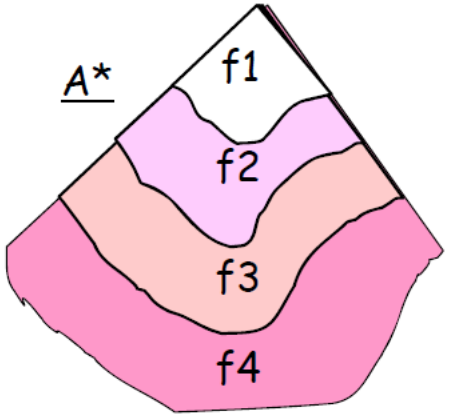
- Iterative Deepening A*
- Simple Memory-bounded A*
- Bi-Directional A*
- Modified Bi-Directional A*
- Dynamic-Weighted A*
- Beam A*

Memory Problems with A*

- A* is similar to breadth-first:



Expand by depth-layers



Expands by f-contours

Memory Bounded Heuristic Search

- How can we solve the memory problem for A* search?
- Idea: Try something like depth first search, but not forgetting everything about the branches that have partially explored.
- Remember the best f-value observed so far in the branch that was about to be deleted.

Iterative Deepening A*

- Each iteration is a depth-first search, just like regular iterative deepening
- Each iteration is not an A* iteration:
  otherwise, still $O(b^m)$ memory
- Use limit on cost (f), instead of depth limit as in regular iterative deepening

Simplified Memory-bounded A*

- Uses all available memory
- Basic idea:
  - Do A* until you run out of memory
  - Throw away node with highest f cost
    - Store f-cost in ancestor node
    - Expand node again if all other nodes in memory are worse

SMA* Search Performance

- Complete if can store all nodes whose f-value are smaller than that of goal in memory
- Finds best solution (and recognizes it), under the same condition.

Question?

THANK YOU